Scheduling jobs with quadratic load cost

Klevis Imeri

The input to the problem:

$$\mathcal{I} := \left\{ \begin{array}{ll} N := \{\text{jobs}\}, & |N| = n, \\ P[i] := \text{processing time of job } i, & |P| = n, \quad P[i] > 0, \\ M := \{\text{machines}\}, & |M| = m. \end{array} \right\}.$$

The exercise allows $P[i] \geq 0$, but zero-processing-time jobs do not affect the cost. We set them aside and assign them to any machine at the end, so we assume $P[i] > 0$. If all jobs have processing time zero, any schedule is optimal.

The output is a schedule $\mathcal{S} = (\mathcal{S}[k])_{k \in M}$, where

$$\mathcal{S}[k] := \{i \in N \mid \text{job } i \text{ is scheduled on machine } k\}.$$

$$\begin{aligned} \text{feasible}(\mathcal{S}) := & \bigcup_{k \in M} \mathcal{S}[k] = N \\ & \wedge \ \forall k, \ell \in M : \quad k \neq \ell \implies \mathcal{S}[k] \cap \mathcal{S}[\ell] = \varnothing. \end{aligned}$$

The cost function is

$$L_k := \sum_{i \in \mathcal{S}[k]} P[i], \qquad f(\mathcal{S}) := \sum_{k=1}^{m} \left(\sum_{i \in \mathcal{S}[k]} P[i] \right)^2 = \sum_{k=1}^{m} L_k^2.$$

We seek

$$\text{OPT}(\mathcal{I}) \in \underset{\mathcal{S}: \text{ feasible}(\mathcal{S})}{\arg\min} \ f(\mathcal{S}).$$

Goal 1. For a fixed constant $C \in \mathbb{N}$, assume

$$\big|\{P[i] \mid i \in N\}\big| \leq C.$$

Our goal is to design an algorithm $\mathcal{A}_C$ with $\mathcal{S}_{\mathcal{A}_C} = \mathcal{A}_C(\mathcal{I})$ such that:

1. $\mathcal{A}_C$ terminates.
2. $\text{feasible}(\mathcal{S}_{\mathcal{A}_C})$.
3. $\text{time}_{\mathcal{A}_C}(\mathcal{I}) \leq \text{poly}(|\mathcal{I}|)$.
4. $f(\mathcal{S}_{\mathcal{A}_C}) = f(\text{OPT}(\mathcal{I}))$.

One may think this problem is very similar to the exact bin packing algorithm in the lecture, where we assumed:

1. A constant number of different sizes.
2. Every item has size at least ε, for a fixed $\varepsilon > 0$.

However, here we have at most C different processing times and $P[i] > 0$, but no fixed positive lower bound, so the second assumption does not hold in general ($P[i]$ can be very small, allowing many jobs on one machine, so we cannot bound their number by a constant as in the lecture). Moreover, our machines have no capacity bound, so even a positive lower bound on processing times would not give a constant bound on the number of jobs per machine.

A machine can contain at most n jobs. Thus, by the stars and bars method, the number of possible configurations is at most

$$R = \binom{n+C}{C} = O(n^C).$$

This is polynomial for fixed C, but R is no longer constant. This is a problem since, similarly to the lecture, at most n nonempty machines are needed, so the total number of combinations of machine configurations is at most

$$\binom{n+R}{R} \leq (n+R)^R = n^{O(n^C)}.$$

This bound is not polynomial, so the lecture's enumeration does not give a polynomial running-time guarantee here.

Let us therefore analyze some properties of how squares behave. For a fixed total load, the sum of squared loads is minimized when all machines have the same load

$$\overline{L} := \frac{\sum_{i \in N} P[i]}{m}.$$

This is quite intuitive, but to put it precisely, write

$$L_k = \overline{L} - (\overline{L} - L_k).$$

The sum of all deviations from the average is zero:

$$\sum_{k=1}^{m} (L_k - \overline{L}) = \sum_{i \in N} P[i] - m\overline{L} = 0.$$

Thus,

$$\sum_{k=1}^{m} L_k^2 = m\overline{L}^2 + 2\overline{L} \sum_{k=1}^{m} (L_k - \overline{L}) + \sum_{k=1}^{m} (L_k - \overline{L})^2 = m\overline{L}^2 + \sum_{k=1}^{m} (L_k - \overline{L})^2 \geq m\overline{L}^2, \qquad (1)$$

with equality exactly when every $L_k = \overline{L}$. Since jobs cannot be split, this may not be possible. We therefore want the machine loads to be close to this average, in the sense of minimizing

$$\sum_{k=1}^{m} L_k^2 = m\overline{L}^2 + \sum_{k=1}^{m} (L_k - \overline{L})^2.$$

Solution using dynamic programming. Intuitively, our lower bound is optimal in the continuous (fractional) setting, but our choices are discrete. We are therefore looking for a discrete schedule whose cost is as close as possible to this fractional optimum. This resembles the distinction between LP and ILP, or between fractional and 0/1 knapsack from the lecture, where we used dynamic programming.

It is not clear that going from LP to ILP would give us an exact solutoin. Let us also try to construct a dynamic programming algorithm for this problem.

To do that, we need a choice to iterate over and a subproblem whose saved solution lets us reconstruct the current solution. We know from above that there are at most R possible configuratoins for one machine. Let our choice be the configuration assigned to the last machine. The subproblem then has the remaining job-count vector minus the chosen configuration, and one fewer machine. Thus, we add one machine at each stage and consider all possible configurations for it.

As part of our dynamic programming algorithm, we construct P' from P by keeping its distinct processing times:

$$P' = (p'_1, \ldots, p'_c), \qquad \{p'_1, \ldots, p'_c\} = \{P[i] \mid i \in N\}, \qquad c \leq C.$$

Jobs with the same processing time belong to the same type.

Given our reasoning above, our recurrence is

$$\mathrm{DP}[k, U] = \min_{0 \leq I \leq U} \left\{ \mathrm{DP}[k-1, U-I] + (I^\mathsf{T} P')^2 \right\}.$$

$$\begin{aligned} k &: \text{ number of machines,} \\ P' = (p'_1, \ldots, p'_c) &: \text{ vector of processing times, one per job type,} \\ U = (u_1, \ldots, u_c) &: \text{ vector counting jobs of each type to schedule,} \\ I = (i_1, \ldots, i_c) &: \text{ vector counting jobs of each type on the last machine,} \\ \mathrm{DP}[k, U] &: \text{ minimum cost of scheduling } U \text{ on } k \text{ machines.} \end{aligned}$$

The vector inequalities are componentwise. We use the base cases

$$\mathrm{DP}[0, \mathbf{0}] = 0, \qquad \mathrm{DP}[0, U] = +\infty \quad (U \neq 0).$$

For reconstuction, we save a minimizing configuration I^* in $\mathrm{CH}[k][U]$ for each state. Below, we give the exact algorithm.

Now we reason about our four goals and prove that our algorithm has these properties.

1. Termination. There is no recursion, and all loops and configuration choices range over finite sets. Therefore, the algorithm terminates.
2. Feasibility.
   (a) During reconstruction, we assign the jobs specified by I to machine k and update $U \leftarrow U - I$. The reconstruction must reach $\mathrm{DP}[0][\mathbf{0}]$ because reaching $\mathrm{DP}[0][U]$ with $U \neq \mathbf{0}$ would give cost $+\infty$. Our algorithm would only choose an infinite cost if every candidate in the arg minimum had infinite cost. This cannot happen because we consider all $I \leq U$, including $I = U$, which gives

   $$\mathrm{DP}[k-1][\mathbf{0}] + (U^\mathsf{T} P')^2 = 0 + (U^\mathsf{T} P')^2 < +\infty.$$

   Thus, no jobs remain unassigned, and

   $$\bigcup_{k \in M} \mathcal{S}[k] = N.$$

   (b) Each machine receives only unassigned jobs. Therefore,

   $$\forall k, \ell \in M: \quad k \neq \ell \implies \mathcal{S}[k] \cap \mathcal{S}[\ell] = \varnothing.$$

Algorithm 1 *Exact-Scheduling*$_C(\mathcal{I})$

Construct P' and U^{all}, the counts of jobs of each type
Let DP, CH
DP[0][$\mathbf{0}$] $\leftarrow 0$
for each job-count vector $\mathbf{0} \neq U \leq U^{\text{all}}$ **do**
    DP[0][U] $\leftarrow +\infty$
end for
for $k = 1, \ldots, m$ **do**
    for each job-count vector $\mathbf{0} \leq U \leq U^{\text{all}}$ **do**
        $I^* \leftarrow \text{CH}[k][U] \leftarrow \arg\min_{I \leq U} \left\{\text{DP}[k-1][U-I] + (I^\mathsf{T} P')^2\right\}$
        DP[k][U] $\leftarrow$ DP[$k-1$][$U - I^*$] $+ ((I^*)^\mathsf{T} P')^2$
    end for
end for
Let $\mathcal{S}$; $U \leftarrow U^{\text{all}}$
for $k = m, m-1, \ldots, 1$ **do**
    $I \leftarrow \text{CH}[k][U]$
    $\mathcal{S}[k] \leftarrow$ any unassigned jobs matching configuration I
    $U \leftarrow U - I$
end for
return $\mathcal{S}$

Both conditions hold, so feasible($\mathcal{S}_{\mathcal{A}_C}$).

3. Running time. Constructing P' and U^{all} takes $O(nC) = O(n)$ time for fixed C. Each component of a job-count vector has at most $n+1$ possible values, including zero, and there are at most C components. Thus, there are at most $(n+1)^C$ job-count vectors. Using arrays, the table sizes, measured in stored numbers, satisfy

$$|\text{DP}| \leq (m+1)(n+1)^C, \qquad |\text{CH}| \leq C(m+1)(n+1)^C,$$

because each entry of CH stores a configuration vector. The base-case initialization fills at most $(n+1)^C$ states.
In the DP loops, we consider m machine counts and at most $(n+1)^C$ job-count vectors. For each state, the arg minimum considers at most $(n+1)^C$ configurations, each taking $O(C)$ time for vector operations, table access, and the scalar product. Thus, the DP computation takes

$$O\big(mC(n+1)^{2C}\big)\,.$$

Initializing reconstruction takes $O(C)$. Reconstruction takes $O(m(n+C)) = O(mn)$ for fixed C, by scanning the jobs for each machine and selecting the required unassigned jobs. Overall,

$$T = O\big(nC + (m+1)C(n+1)^C + mC(n+1)^{2C} + mn\big) = O\big(mC(n+1)^{2C}\big)\,,$$

which is poly($|\mathcal{I}|$) for fixed C.

4. Optimal solution. We have to prove that our solution is one of the optimal solutions. As usual with dynamic programming, we prove that the DP invariant holds. We prove by induction on k that, for every $\mathbf{0} \leq U \leq U^{\text{all}}$,

$$\text{DP}[k][U] = \min_{I_1 + \cdots + I_k = U} \sum_{\ell=1}^{k} (I_\ell^\mathsf{T} P')^2,$$

and, for $k \geq 1$, CH$[k][U]$ stores a last-machine configuration attaining this minimum together with an optimal schedule for the remaining jobs.

Base case. DP$[0][\mathbf{0}] = 0$ is optimal because there are no jobs, so no cost. For $U \neq \mathbf{0}$, no schedule on zero machines exists, so DP$[0][U] = +\infty$.

Inductive step. Assume the invariant holds for $k-1$ and every job-count vector. Any schedule on k machines assigns some configuration $I \leq U$ to its last machine, leaving $U - I$ for the other $k-1$ machines. By the induction hypothesis, their minimum cost is DP$[k-1][U-I]$. Conversely, combining such an optimal schedule with configuration I gives cost

$$\text{DP}[k-1][U-I] + (I^{\mathsf{T}} P')^2.$$

Our algorithm considers every possible $I \leq U$ and selects a minimum, so DP$[k][U]$ is optimal and CH$[k][U]$ stores a minimizing last-machine configuration. Thus, the invariant holds.

The reconstruction follows these stored minimizing configurations, so its total cost is DP$[m][U^{\text{all}}]$. Therefore,

$$f(\mathcal{S}_{\mathcal{A}_C}) = \text{DP}[m][U^{\text{all}}] = f(\text{OPT}(\mathcal{I})).$$

□

Solution using enumeration. I wrote this solution later because I did not see it immediately. I tried to bound the enumeration for a long time without success, until I had the idea of considering pairs of configurations with the same parity pattern.

Definition 1 (Parity pattern)**.**

$$I = (i_1, \ldots, i_c) \longmapsto \pi(I) := (i_1 \bmod 2, \ldots, i_c \bmod 2) \in \{0,1\}^c.$$

$$0: \text{ even entry}, \quad 1: \text{ odd entry}.$$

Thus, each entry contributes an even/odd choice, giving 2^c parity patterns.

Lemma 1. *$\exists$ an optimal schedule using at most $2^c \leq 2^C$ distinct configurations.*

Proof. By contradiction. Among all optimal schedules, choose one minimizing

$$\text{tie-breaker} := \sum_{k=1}^{m} \sum_{j=1}^{c} (I_{k,j})^2,$$

where $I_{k,j}$ is the number of type-j jobs on machine k. Suppose

$$\text{the chosen schedule uses more than } 2^c \text{ distinct configurations.} \tag{2}$$

Since there are only 2^c parity patterns, two different configurations $A = (a_1, \ldots, a_c)$ and $B = (b_1, \ldots, b_c)$ have the same pattern. Thus, $(A+B)/2$ has integer entries because corresponding entries have the same parity (even + even is even, and odd + odd is even). Replace one machine of each configuration by two machines both using $(A+B)/2$. The total job counts remain $A + B$. Since

$$(A^{\mathsf{T}} P')^2 + (B^{\mathsf{T}} P')^2 - 2\left(\frac{A^{\mathsf{T}} P' + B^{\mathsf{T}} P'}{2}\right)^2 \geq 0,$$

the cost cannot increase:

$$2\left(\frac{A^\mathsf{T}P' + B^\mathsf{T}P'}{2}\right)^2 \le (A^\mathsf{T}P')^2 + (B^\mathsf{T}P')^2.$$

The cost strictly decreases if $A^\mathsf{T}P' \neq B^\mathsf{T}P'$ and stays unchanged if $A^\mathsf{T}P' = B^\mathsf{T}P'$. Hence, the resulting schedule is still optimal. However, the squared job counts decrease by

$$\sum_{j=1}^{c}\left(a_j^2 + b_j^2 - 2\left(\frac{a_j + b_j}{2}\right)^2\right) = \frac{1}{2}\sum_{j=1}^{c}(a_j - b_j)^2 > 0,$$

since $A \neq B$. This contradicts our choice of an optimal schedule with the smallest tie-breaker, so assumption (2) is false. □

We therefore enumerate which at most 2^c configurations are used and how many machines use each configuration.

Algorithm 2 *Exact-Scheduling*$_C(\mathcal{I})$

```
Construct P' = (p'_1, ..., p'_c), the distinct processing times
U^all ← (|{i ∈ N : P[i] = p'_j}|)_{j=1}^c
R ← {I : 0 < I ≤ U^all}
S_min ← the schedule assigning all jobs to machine 1
for each H ⊆ R with |H| ≤ 2^c do
    for each vector X = (x_I)_{I∈H} ∈ {1, ..., min(m, n)}^{|H|} do
        if Σ_{I∈H} x_I ≤ m and Σ_{I∈H} x_I I = U^all then
            Construct S using x_I machines with configuration I for each I ∈ H
            if f(S) < f(S_min) then
                S_min ← S
            end if
        end if
    end for
end for
return S_min
```

I : vector counting jobs of each type on one machine,
H : set of distinct configurations used,
X : vector of machine counts, one entry per configuration in H,
x_I : number of machines using configuration I.

We now prove the required properties of the enumeration algorithm.

1. Termination. $\mathcal{R}$ is finite, so its subsets H are finite in number. For each H, the possible vectors X are also finite in number. Thus, all loops are finite and the algorithm terminates.
2. Feasibility. We construct a schedule only when

$$\sum_{I\in H} x_I \le m, \qquad \sum_{I\in H} x_I I = U^{\text{all}}.$$

The first condition ensures that we use at most m machines. The second ensures that we assign exactly the available number of jobs of each type. Assigning distinct jobs to the required positions gives

$$\bigcup_{k\in M} \mathcal{S}[k] = N, \qquad \forall k \neq \ell :\ \mathcal{S}[k] \cap \mathcal{S}[\ell] = \varnothing.$$

The initial $\mathcal{S}_{\min}$ is feasible and is updated only with feasible schedules, so the returned schedule is feasible.

3. Running time. Lines 1–2 take $O(nc) = O(n)$ for fixed C. In line 3, each configuration has c components with at most $n+1$ possible values each, including zero. Therefore,

$$|\mathcal{R}| + 1 \leq (n+1)^c,$$

and constructing $\mathcal{R}$ takes $O(c(n+1)^c)$ time. Line 4 takes $O(n+m)$ time.
The loop in line 5 considers at most

$$\sum_{r=0}^{2^c} \binom{|\mathcal{R}|}{r} \leq (|\mathcal{R}|+1)^{2^c} \leq (n+1)^{c2^c}$$

sets H. For each H, the loop in line 6 considers at most

$$\min(m,n)^{|H|} \leq (n+1)^{2^c}$$

vectors X. Checking the conditions takes $O(c|H|) \leq O(c2^c)$. Constructing a feasible schedule, evaluating its cost, and updating $\mathcal{S}_{\min}$ take $O(n+m)$ for fixed C. Thus, for fixed C,

$$T = O\Big((n+m)(n+1)^{(C+1)2^C}\Big) \subseteq O\Big((n+m)^{(C+2)2^C}\Big) \in \text{poly}(|\mathcal{I}|).$$

4. Optimal solution. By Lemma 1, there exists an optimal schedule with the smallest tie-breaker that uses at most 2^c distinct configurations. Our enumeration considers its set H of nonempty configurations and its vector X of machine counts. Thus, it examines a schedule with optimal cost. Since we keep the minimum-cost schedule and every candidate is feasible, the returned schedule is optimal:

$$f(\mathcal{S}_{\min}) = f(\text{OPT}(\mathcal{I})).$$

□

Goal 2. Our goal is to design an algorithm $\mathcal{A}_\varepsilon$ with $\mathcal{S}_{\mathcal{A}_\varepsilon} = \mathcal{A}_\varepsilon(\mathcal{I})$, such that:

1. $\mathcal{A}_\varepsilon$ terminates.
2. feasible($\mathcal{S}_{\mathcal{A}_\varepsilon}$).
3. $\text{time}_{\mathcal{A}_\varepsilon}(\mathcal{I}) \leq \text{poly}(|\mathcal{I}|)$ for fixed ε.
4. $f(\mathcal{S}_{\mathcal{A}_\varepsilon}) \leq (1+\varepsilon) f(\text{OPT}(\mathcal{I}))$, $\forall \varepsilon > 0$ ($\mathcal{A}_\varepsilon$ is a PTAS).

As we did in class with the knapsack FPTAS, we will try rounding the processing times.

Algorithm 3 *Maybe-PTAS-Scheduling*$(\mathcal{I}, \varepsilon)$

1: Choose $K \in (0, ?]$
2: **for** $i \in N$ **do**
3: $\quad \widehat{P}[i] \leftarrow K \left\lfloor \dfrac{P[i]}{K} \right\rfloor$
4: **end for**
5: $\widehat{\mathcal{I}} \leftarrow \{N, \widehat{P}, M\}$
6: $C \leftarrow \left|\{\widehat{P}[i] \mid i \in N\}\right|$
7: **return** *Exact-Scheduling*$_C(\widehat{\mathcal{I}})$

Notice the choice of K. In the lecture, for knapsack, we chose

$$K \leftarrow \max\left\{1, \left\lfloor \frac{\varepsilon}{n} p_{\max} \right\rfloor\right\}.$$

Here, we cannot impose $K \geq 1$ because all processing times could be smaller than 1, in which case rounding would make them all zero. We must therefore allow K to be smaller than 1, while keeping $K > 0$ because we divide by it. The symbol ? means that we have not yet chosen the upper bound on K because we will determine it during the proof to meet the PTAS requirements.

Also, unlike knapsack from class, where we maximize profit, here we minimize the sum of squared machine loads. We use the standard inequality step, similar to knapsack in the lecture, where for maximization we had

$$c(\mathrm{OPT}) \geq c(\mathrm{OPT}') \geq c'(\mathrm{OPT}') \geq c'(\mathrm{OPT}).$$

Here, for minimization, we have

$$c'(\mathrm{OPT}') \leq c'(\mathrm{OPT}) \leq c(\mathrm{OPT}) \leq c(\mathrm{OPT}').$$

In our notation,

$$\widehat{f}(\mathcal{S}_{\mathcal{A}_\varepsilon}) \leq \widehat{f}(\mathrm{OPT}) \leq f(\mathrm{OPT}) \leq f(\mathcal{S}_{\mathcal{A}_\varepsilon}).$$

Here, $\widehat{f}$ is the cost computed using the rounded processing times. Either way, the idea is to bound the error by $\varepsilon f(\mathrm{OPT})$. Thus, we need to show

$$\underbrace{f(\mathcal{S}_{\mathcal{A}_\varepsilon}) - \widehat{f}(\mathcal{S}_{\mathcal{A}_\varepsilon}) \leq \varepsilon f(\mathrm{OPT})}_{\text{sufficient error bound}} \tag{3}$$

$$\iff \quad f(\mathcal{S}_{\mathcal{A}_\varepsilon}) \leq \widehat{f}(\mathcal{S}_{\mathcal{A}_\varepsilon}) + \varepsilon f(\mathrm{OPT}).$$

Since $\mathcal{S}_{\mathcal{A}_\varepsilon}$ minimizes the rounded cost,

$$\begin{aligned} f(\mathcal{S}_{\mathcal{A}_\varepsilon}) &\leq \widehat{f}(\mathcal{S}_{\mathcal{A}_\varepsilon}) + \varepsilon f(\mathrm{OPT}) \\ &\leq \widehat{f}(\mathrm{OPT}) + \varepsilon f(\mathrm{OPT}) \\ &\leq f(\mathrm{OPT}) + \varepsilon f(\mathrm{OPT}) \\ &= (1+\varepsilon) f(\mathrm{OPT}). \end{aligned}$$

It is worth mentioning that, since we round down, $\widehat{f}(\mathcal{S}_{\mathcal{A}_\varepsilon}) \leq f(\mathcal{S}_{\mathcal{A}_\varepsilon}) \implies f(\mathcal{S}_{\mathcal{A}_\varepsilon}) - \widehat{f}(\mathcal{S}_{\mathcal{A}_\varepsilon}) \geq 0$.

So now, the only thing left to prove for the approximation guarantee is the sufficient error bound in equation (3).

For a generic $K > 0$, write $S = \mathcal{S}_{\mathcal{A}_\varepsilon}$. The load error on each machine satisfies

$$0 \leq L_k - \widehat{L}_k = \sum_{i \in S[k]} \left(P[i] - K \left\lfloor \frac{P[i]}{K} \right\rfloor \right) \leq |S[k]| K \leq nK.$$

Thus,

$$\begin{aligned} f(S) - \widehat{f}(S) &= \sum_{k=1}^{m} (L_k - \widehat{L}_k)(L_k + \widehat{L}_k) \\ &\leq nK \sum_{k=1}^{m} (L_k + \widehat{L}_k) \\ &\leq 2nK \sum_{k=1}^{m} L_k = 2nK \sum_{i \in N} P[i]. \end{aligned}$$

We will choose K so that this bound is at most $\varepsilon f(\mathrm{OPT})$. From equation (1) above, we know

$$f(\mathrm{OPT}) \geq m\overline{L}^2.$$

Thus, for $\overline{L} > 0$, a sufficient choice is

$$2nKm\overline{L} \leq \varepsilon m\overline{L}^2 \iff K \leq \frac{\varepsilon\overline{L}}{2n}.$$

Taking $K = \frac{\varepsilon\overline{L}}{2n}$ therefore gives the required approximation guarantee.

The problem is the running time. With $P_{\max} := \max_{i \in N} P[i] \leq m\overline{L}$, the number of rounded processing-time types satisfies

$$C \leq 1 + \left\lfloor \frac{P_{\max}}{K} \right\rfloor \leq 1 + \frac{P_{\max}}{K} \leq 1 + \frac{m\overline{L}}{\varepsilon\overline{L}/(2n)} = 1 + \frac{2nm}{\varepsilon}.$$

This bound depends on n and m, even for fixed ε. Our exact algorithm has running time $O(mC(n+1)^{2C})$, which is polynomail only for fixed C. Even if we could improve the bound to $P_{\max} \leq a\overline{L}$ for a constant a, we would still have the n in the denominator of K, giving

$$C \leq 1 + \frac{a\overline{L}}{\varepsilon\overline{L}/(2n)} = 1 + \frac{2an}{\varepsilon}.$$

So the dependence on n would still cause problems. Thus, this analysis gives the approximatoin guarantee but does not give a polynomial running-time guarantee. We need a better rounding or grouping method that controls the error while keeping the number of types dependent only on ε.

Let us step back and see what the bigger problem could be. We allowed each of the n jobs to contribute up to K to the rounding error, giving a load error bounded by nK, as in knapsack from the lecture. Here, however, both our DP and our enumeration algorithm have C in the exponent. In other words, controlling the acumulated error forced too many rounded types.

This happens because processing times can be very small compared with $\overline{L}$, so a machine may contain many jobs. Allowing a rounding error of up to K per job makes these errors accumulate. Our bound therefore requires a very small rounding step K, which allows too many different rounded processing times.

The first idea is that, because we want to minimize $\sum_{k=1}^{m}(L_k - \overline{L})^2$, if $P[i] \geq \overline{L}$, we can assign job i to a machine on its own in an optimal schedule. By intuition, we would rather use the other jobs to bring less loaded machines closer to $\overline{L}$ than add them to this machine and move its load even farther above $\overline{L}$.

For the formal proof, suppose an optimal schedule puts other jobs of total load $x > 0$ on the machine containing job i. This moves its load from $P[i] \geq \overline{L}$ to $P[i] + x$, farther above the average. Since this machine is above the average, another machine has load $b < \overline{L}$. We can instead put those other jobs on that less loaded machine. The total squared distance from the average decreases, because

$$\begin{aligned}&(P[i] + x - \overline{L})^2 + (b - \overline{L})^2 \\ &\quad - \big((P[i] - \overline{L})^2 + (b + x - \overline{L})^2\big) = 2\underbrace{x}_{>0}\ \underbrace{(P[i] - b)}_{>0 \text{ since } P[i] \geq \overline{L} > b} > 0.\end{aligned}$$

Thus, using the other jobs on the less loaded machine gives a better schedule, contradicting optimality.

By setting all jobs with $P[i] \geq \overline{L}$ aside on their own machines, the remaining instance has $P_{\max} < \overline{L}$. We keep the original average $\overline{L}$ fixed as our scale for grouping and rounding. This removes the factor m from our bound on C, giving $C \leq 1 + 2n/\varepsilon$. However, we still have the dependence on n to remove.

Therefore, to remove this dependence, we follow the structure of the general bin packing algorithm from the lecture. So we set small jobs aside, solve a rounded instance exactly, and add the small jobs back using FirstFit.

Let $s > 0$ be a threshold to be chosen during the analysis, and let N' be the current set of jobs remaining after isolating the large jobs. We still need to account for the total load of jobs with $P[i] < s$ when solving the rounded instance. We therefore replace these jobs temporarily by

$$q = \left\lceil \frac{\sum_{\substack{i \in N' \\ P[i] < s}} P[i]}{s} \right\rceil$$

identical placeholder jobs of processing time s. These reserve space for the small jobs without deciding in advance which original jobs go on which machine.

Algorithm 4 *PTAS-Scheduling*$(\mathcal{I}, \varepsilon)$

$\overline{L} \leftarrow \frac{1}{m} \sum_{i \in N} P[i]$
$\eta \leftarrow \min\{1, \sqrt{1+\varepsilon} - 1\}$; $K \leftarrow \frac{\eta^2}{8} \overline{L}$
Isolate every job with $P[i] \geq \overline{L}$ on its own machine
if all jobs have been assigned **then return** the resulting schedule
$N' \leftarrow$ unassigned jobs; $M' \leftarrow$ unused machines
$s \leftarrow \frac{\eta}{4} \overline{L}$
for each $i \in N'$ with $s \leq P[i] < \overline{L}$ **do**
    $\widehat{P}[i] \leftarrow K \left\lceil \frac{P[i]}{K} \right\rceil$
end for
$q \leftarrow \left\lceil \frac{\sum_{\substack{i \in N' \\ P[i] < s}} P[i]}{s} \right\rceil$
In N', replace jobs with $P[i] < s$ by q placeholders, each with $\widehat{P} = s$
$C \leftarrow \left| \{\widehat{P}[i] \mid i \in N'\} \right|$
$\widehat{\mathcal{S}} \leftarrow$ *Exact-Scheduling*$_C(\widehat{\mathcal{I}} = \{N', \widehat{P}, M'\})$
$\widehat{L}_k \leftarrow \sum_{i \in \widehat{\mathcal{S}}[k]} \widehat{P}[i]$
Remove the placeholders and restore the original processing times
Add back the original jobs with $P[i] < s$ using FirstFit on M',
    with load limit $\widehat{L}_k + s$ on machine k
return the resulting schedule together with the isolated jobs

The choices of K and s are justified during the approximation analysis.

1. Termination. All loops and sums range over finite sets, and *Exact-Scheduling* terminates by Goal 1. FirstFit considers finitely many jobs and machines. Therefore, the algorithm terminates.

2. Feasibility. The isolated jobs are assigned to their own machines. The remaining jobs with $P[i] \geq s$ are assigned feasibly by *Exact-Scheduling*, and restoring their original processing times can only decrease the loads.
   The jobs with $P[i] < s$ are added back using FirstFit with load limits $\widehat{L}_k + s$, which allow all small jobs to fit. Each original job is assigned exactly once to a machine in M, so the resulting schedule is feasible.
3. Running time. All steps except line 13 take $O(nm)$ time. We still need to analyze the call to *Exact-Scheduling* in line 13. We use the dynamic programming algorithm because it has a smaller exponent. Its running time was

$$O\big(mC(n+1)^{2C}\big)\,.$$

   The modified instance has at most n jobs, since q is at most the number of original jobs with $P[i] < s$. We only need to show that C depends on ε alone.
   Let us analyze the number of distinct processing times. All q placeholders have processing time s, so together they contribute at most one type. The other jobs satisfy $s \leq P[i] < \overline{L}$ and are rounded up to multiples of K. Hence,

$$C \leq 1 + \left\lceil \frac{\overline{L}}{K} \right\rceil .$$

   We can therefore deduce one condition on our choice of K: choose

$$K = \alpha(\varepsilon)\overline{L}, \qquad \alpha(\varepsilon) > 0.$$

   Then $C \leq 1 + \lceil 1/\alpha(\varepsilon) \rceil$ depends only on ε. We will determine $\alpha(\varepsilon)$ during the approximation analysis. Given such a choice of K, the total running time is

$$O\big(nm + mC(n+1)^{2C}\big) \in \text{poly}(|\mathcal{I}|) \qquad \text{for every fixed } \varepsilon > 0.$$

4. Approximation. We now show that our algorithm is a PTAS. Start with an optimal schedule OPT for the original instance, with the large jobs isolated as proved above. In this analysis, $\widehat{\mathcal{S}}$ also includes the isolated jobs with their unchanged processing times. Similarly to the knapsack proof, we want to bound the error by $\varepsilon f(\text{OPT})$:

$$\underbrace{f(\mathcal{S}_{A_\varepsilon}) - f(\text{OPT})}_{\text{error}} \leq \varepsilon f(\text{OPT}).$$

   Next, we substitute the FirstFit bound $L_k \leq \widehat{L}_k + s$ into the error:

$$\begin{aligned} f(\mathcal{S}_{A_\varepsilon}) - f(\text{OPT}) &\leq \sum_{k=1}^{m} (\widehat{L}_k + s)^2 - f(\text{OPT}) \\ &= \underbrace{\widehat{f}(\widehat{\mathcal{S}}) - f(\text{OPT})}_{\text{rounding and placeholders}} + \underbrace{2s \sum_{k=1}^{m} \widehat{L}_k + ms^2}_{\text{FirstFit}}\,. \end{aligned}$$

   We now bound these terms using rounding and placeholders. For each remaining job with $P[i] \geq s$, rounding up increases its processing time by at most K. Since $P[i] \geq s$, we also have $K \leq \frac{K}{s} P[i]$. Therefore,

$$\widehat{P}[i] = K \left\lceil \frac{P[i]}{K} \right\rceil \leq P[i] + K \leq P[i] + \frac{K}{s} P[i] = \left(1 + \frac{K}{s}\right) P[i].$$

This is one part of the connection between the rounded exact subproblem's cost and the cost of our PTAS algorithm. Let L_k^{OPT} be the load on machine k in OPT, and let

$$t_k = \sum_{\substack{i \in \mathrm{OPT}[k] \\ P[i] < s}} P[i]$$

be its total small-job load. Let $\widehat{L}_k^{\mathrm{OPT}}$ be its load after rounding only the jobs with $P[i] \geq s$. To bound the increase in the combined load on machine k, we sum both sides over its jobs with $P[i] \geq s$:

$$\underbrace{\sum_{\substack{i \in \mathrm{OPT}[k] \\ P[i] \geq s}} \widehat{P}[i]}_{\widehat{L}_k^{\mathrm{OPT}} - t_k} \leq \left(1 + \frac{K}{s}\right) \underbrace{\sum_{\substack{i \in \mathrm{OPT}[k] \\ P[i] \geq s}} P[i]}_{L_k^{\mathrm{OPT}} - t_k}.$$

Now we need to bound the load increase for jobs with $P[i] < s$ using our placeholders. We can distribute the q placeholders so that machine k receives q_k placeholders with

$$\sum_k q_k = q, \qquad q_k s \leq t_k + s.$$

Thus, replacing the small jobs by placeholders adds at most s to each machine's load. All that remains is to put these bounds together. Our comparison schedule has load at most

$$\left(1 + \frac{K}{s}\right)(L_k^{\mathrm{OPT}} - t_k) + t_k + s \leq \left(1 + \frac{K}{s}\right) L_k^{\mathrm{OPT}} + s.$$

The isolated machines satisfy this bound as well, since their loads stay unchanged. Since *Exact-Scheduling* minimizes the rounded cost,

$$\widehat{f}(\widehat{\mathcal{S}}) \leq \sum_{k=1}^{m} \left(\left(1 + \frac{K}{s}\right) L_k^{\mathrm{OPT}} + s\right)^2.$$

Also,

$$f(\mathrm{OPT}) = \sum_{k=1}^{m} (L_k^{\mathrm{OPT}})^2.$$

Subtracting this from both sides gives

$$\widehat{f}(\widehat{\mathcal{S}}) - f(\mathrm{OPT}) \leq \sum_{k=1}^{m} \left[\left(\left(1 + \frac{K}{s}\right) L_k^{\mathrm{OPT}} + s\right)^2 - (L_k^{\mathrm{OPT}})^2\right].$$

Expanding the squares and using $\sum_{k=1}^{m} L_k^{\mathrm{OPT}} = m\overline{L}$ gives

$$\begin{aligned}\widehat{f}(\widehat{\mathcal{S}}) - f(\mathrm{OPT}) \leq &\left(\left(1 + \frac{K}{s}\right)^2 - 1\right) f(\mathrm{OPT}) \\ &+ 2\left(1 + \frac{K}{s}\right) sm\overline{L} + ms^2.\end{aligned}$$

By Equation (1), $m\overline{L}^2 \leq f(\mathrm{OPT})$, and also

$$ms^2 = \frac{s^2}{\overline{L}^2}\, m\overline{L}^2 \leq \frac{s^2}{\overline{L}^2} f(\mathrm{OPT}).$$

Summing the comparison schedule's load bound gives

$$\sum_{k=1}^{m} \widehat{L}_k \le \sum_{k=1}^{m} \left(\left(1 + \frac{K}{s}\right) L_k^{\mathrm{OPT}} + s \right) = \left(1 + \frac{K}{s}\right) m\overline{L} + ms.$$

We now substitute both bounds into the original error decomposition and combine the $sm\overline{L}$ and ms^2 terms:

$$\begin{aligned} f(\mathcal{S}_{A_\varepsilon}) - f(\mathrm{OPT}) &\le \left(\left(1 + \frac{K}{s}\right)^2 - 1 \right) f(\mathrm{OPT}) \\ &\quad + 2\left(1 + \frac{K}{s}\right) sm\overline{L} + ms^2 \\ &\quad + 2s\left(\left(1 + \frac{K}{s}\right) m\overline{L} + ms \right) + ms^2 \\ &= \left(\left(1 + \frac{K}{s}\right)^2 - 1 \right) f(\mathrm{OPT}) + 4\left(1 + \frac{K}{s}\right) sm\overline{L} + 4ms^2 \\ &\le \left[\left(1 + \frac{K}{s} + \frac{2s}{\overline{L}}\right)^2 - 1 \right] f(\mathrm{OPT}), \end{aligned}$$

where the last step again uses $m\overline{L}^2 \le f(\mathrm{OPT})$. Adding $f(\mathrm{OPT})$ to both sides gives

$$f(\mathcal{S}_{A_\varepsilon}) \le \left(1 + \frac{K}{s} + \frac{2s}{\overline{L}}\right)^2 f(\mathrm{OPT}) \le (1+\varepsilon) f(\mathrm{OPT}),$$

where the last inequality holds if we choose K and s such that

$$\left(1 + \frac{K}{s} + \frac{2s}{\overline{L}}\right)^2 \le 1 + \varepsilon.$$

Now the only remaining task is to choose K and s subject to our constraints. The running-time analysis requires $K = \alpha(\varepsilon)\overline{L}$ for some $\alpha(\varepsilon) > 0$. Taking square roots in the approximation condition gives

$$\frac{K}{s} + \frac{2s}{\overline{L}} \le \sqrt{1+\varepsilon} - 1.$$

Let $\eta = \min\{1, \sqrt{1+\varepsilon} - 1\}$. We split this allowance equally:

$$\frac{K}{s} = \frac{\eta}{2}, \qquad \frac{2s}{\overline{L}} = \frac{\eta}{2}.$$

Solving these equations gives

$$s = \frac{\eta}{4}\overline{L}, \qquad K = \frac{\eta^2}{8}\overline{L}, \qquad \alpha(\varepsilon) = \frac{\eta^2}{8} > 0.$$

These choices give $0 < s < \overline{L}$ and

$$\left(1 + \frac{K}{s} + \frac{2s}{\overline{L}}\right)^2 = (1+\eta)^2 \le 1 + \varepsilon.$$

Also, $C \le 1 + \lceil 8/\eta^2 \rceil$ depends only on ε. Thus, the algorithm is a PTAS. $\square$